

Guarded Bulk File Reorganization: System-Directory Detection, Survivable Execution, Reversible Re-grouping, and Safe Delegation to Autonomous Agents

Ian Too 

<https://orcid.org/0009-0000-4888-1941>

Independent Researcher

github.com/ianktoo/tidy-up

Abstract

Tools that reorganize files in bulk are, by construction, capable of destroying an operating system installation: the same loop that sorts a downloads folder into categories will happily scatter `C:\Windows` across a dozen subdirectories. Existing utilities address this weakly or not at all, relying on the user to type the right path. We present the design, implementation and evaluation of a guarded bulk reorganization system built into TIDY-UP, a cross-platform command-line tool written in Rust. Four contributions are described. First, a *pure, platform-parameterized classifier* that judges whether a directory is managed by the operating system, returning a three-level risk verdict with machine-readable reasons; because the target platform is an argument rather than a compile-time constant, all three rule tables (Windows, macOS, Linux) are exercised from a single host, and a 24-entry negative corpus guards against warning fatigue. Second, an *empirical writability probe* that sidesteps the well-known disagreement between POSIX mode bits, access-control lists, read-only mounts and macOS System Integrity Protection by attempting a write and observing the result. Third, a *survival architecture* in which per-item failures are classified, collected and reported rather than propagated, with a single stated exception: a change that cannot be journaled is fatal, because a change that cannot be journaled cannot be undone. Fourth, an *idempotent re-grouping algorithm* that unpacks an arbitrarily nested directory tree and re-files it under an ordered list of grouping keys, computes which directories the operation will empty, and records their removal so that undo restores the original tree exactly, including directories that held nothing. Fifth, an account of *delegating filesystem mutation to an autonomous agent*, which we argue is admissible only where every action is reviewable before it happens and reversible after. We contrast two interface shapes for such delegation, validating the operations an agent names against never letting it name one, and report that the second is strictly stronger, because validation must anticipate every way a path can escape a root while the alternative admits no path at all. Applying that to a Model Context Protocol server yields an interface in which indirect prompt injection has no argument to express itself through. We further report a structural limit of journal-based undo: a journal records transitions, so it can answer neither “what was left untouched” nor “can this still be undone”, both of which a reviewer needs. The implementation adds 10,265 lines across sixteen modules and introduces *zero* new dependencies, keeping the transitive dependency count at 54 crates. An evaluation over 455 automated tests reports the properties each mechanism is designed to guarantee, including a byte-for-byte reorganize/undo round trip, the non-overridability of a genuine permission wall, and the refusal of a plan that reaches outside its own root.

Keywords: file system safety; bulk reorganization; cross-platform systems; capability security; agent tool design; prompt injection; idempotence; reversible operations; observability; Rust.

1 Introduction

A bulk file organizer reads a directory, decides where each file belongs, and moves it there. The mechanism is simple and the failure mode is severe. Nothing in the mechanism distinguishes a downloads folder from a system directory: both are directories full of files, and the loop that tidies one will tidy the other. A user who types the wrong path, a script that interpolates an empty variable into `organize "$DIR"`, or an administrator experimenting with elevated privileges can, in a single command, relocate the contents of an operating system installation into category subdirectories.

The conventional defences are inadequate in a specific and instructive way. Dry-run modes, confirmation prompts, refusal to overwrite, and undo journals all protect the user against

mistakes made *inside* a correctly chosen directory. None of them protects against choosing the wrong directory in the first place. An undo journal in particular offers false comfort: replaying it restores file locations, but it cannot restore a system that failed to boot in the interim, and it cannot help at all if the files that moved were the ones the undo procedure itself depends on.

This paper describes what we added to TIDY-UP, an existing open-source cross-platform organizer, to close that gap, together with three further mechanisms that the gap analysis exposed as prerequisites. Section 2 situates the work. Section 3 gives the system architecture. Section 4 presents the detection classifier, which is the principal contribution. Section 5 treats permission determination, Section 6 the survival architecture, Section 7 the re-grouping algorithm, and Section 8 observability. Section 10 evaluates the result and Section 11 discusses its limits.

1.1 Problem statement

Let D be the set of directories on a machine. A bulk reorganizer applies a transformation $T : D \rightarrow D$ that relocates the regular files beneath a root $r \in D$. We require a predicate

$$\text{risk} : D \rightarrow \{\text{Safe}, \text{Caution}, \text{Dangerous}\}$$

and a policy that maps a risk level, an access intent, and a set of user-supplied flags to one of $\{\text{proceed silently}, \text{warn}, \text{warn and confirm}, \text{refuse}\}$. Four properties are desirable, and they are in tension:

P1 (Soundness, informal). Directories the operating system manages are not classified **Safe**.

P2 (Precision). Ordinary user directories are classified **Safe**. A system that warns about `D:\Downloads` trains users to bypass it, destroying P1 in practice.

P3 (Testability). The rules for each platform can be validated without access to that platform and without touching real system directories.

P4 (Escapability). A user who genuinely intends to operate on a flagged directory can, except where the operating system itself refuses.

P2 deserves emphasis. A guard is a human-factors artifact as much as a technical one. Warning fatigue is the dominant failure mode of safety mechanisms in interactive tools, and a false positive is therefore not a cosmetic defect but a direct attack on the mechanism's purpose. This observation drove two design decisions reported in Section 4.5 that deliberately reduce coverage.

1.2 Contributions

1. A pure, platform-parameterized system-directory classifier with machine-readable reasons and per-reason severity, validated by a table-driven corpus that exercises all three platforms from one host (Section 4).
2. An empirical writability probe that is correct in the presence of ACLs, read-only mounts and macOS System Integrity Protection, and a corresponding policy rule that a permission wall is never overridable (Section 5).
3. A survival architecture with a classified failure taxonomy, a single documented fatal exception, and grouped, actionable end-of-run reporting (Section 6).
4. An idempotent, reversible re-grouping algorithm, including the computation of directories emptied by a run and their restoration on undo (Section 7).

5. An account of delegating filesystem mutation to an autonomous agent: a machine interface whose exit codes separate a refusal from a failure, the argument that an interface in which an agent cannot name a path is strictly stronger than one that validates the paths it names, and a configuration model in which discovered settings may lower the bar but never raise it (Section 9).
6. The observation that journal-based undo cannot, structurally, answer the two questions a reviewer most needs, and what has to be added to it (Section 9.4).
7. An evaluation, and an account of the engineering constraint — zero new dependencies — that shaped every one of these designs (Section 10).

2 Background and related work

2.1 Guarded destructive operations

The idea of refusing a dangerous argument is old and its canonical instance is instructive. GNU `rm` implements `-preserve-root`, enabled by default, which refuses to operate recursively on `/`. The design is notable for what it does *not* attempt: it protects exactly one path, by exact comparison, and makes no judgement about `/usr` or `/home`. This is a deliberate trade toward precision (P2) at the expense of soundness (P1), and the reasoning is sound for a general-purpose tool invoked by scripts thousands of times a day.

A bulk reorganizer occupies a different point on that curve. It is invoked interactively, rarely, on a directory the user names explicitly, and its blast radius is an entire subtree. The cost of a false positive is one extra keystroke; the cost of a false negative is an unbootable machine. We therefore extend coverage well beyond the filesystem root while treating every extension as a precision risk to be tested against a negative corpus.

Package managers offer a second reference point. Both `dpkg` and `rpm` maintain a database of files owned by the system and refuse to let an installation overwrite them. That approach is sound and precise, but it presumes a curated database, which does not exist on Windows or macOS and which a third-party tool cannot construct. Our classifier approximates that database with a static rule table plus runtime facts.

2.2 Capability determination

Determining whether a process may write to a directory is a recurring difficulty. The POSIX `access(2)` interface answers with respect to the *real* user ID, which is usually not the question being asked; `faccessat` with `AT_EACCESS` corrects that, but neither call is authoritative. The manual page for `access(2)` explicitly warns that its result may be invalidated between the check and the subsequent operation, a classic time-of-check-to-time-of-use hazard, and both calls can disagree with the filesystem on NFS, with POSIX and NFSv4 access-control lists, on read-only mounts, and under macOS System Integrity Protection, where even a process running as `root` is denied. On Windows the mode-bit model does not apply at all, and `FILE_ATTRIBUTE_READONLY` on a directory does not mean the directory is read-only: it signals a customized folder view and is set on entirely ordinary directories such as `Documents`.

The technique of resolving this by attempting the operation is folklore in systems programming rather than a published result. We adopt it, scope it carefully so that it never runs when the caller promised not to write, and report its behaviour.

2.3 Capability security and agent tools

The distinction drawn in Section 9.2 between validating a name and holding an unforgeable reference is the capability-security distinction, which is old [1] and was developed at length by

Miller [2]. The confused-deputy problem [3] is the canonical failure of the alternative: authority exercised on behalf of a caller who should not have had it, because the request named a resource rather than presenting a right to it.

What is comparatively new is the caller. Tool-using language models introduce a caller that composes requests from text it was given, and that text may be adversarial without the operator knowing: the indirect prompt injection setting described by Greshake and colleagues [4]. Our contribution here is not the principle but a worked application of it to a destructive filesystem tool, and the observation that a plan/execute split maintained for testability turns out to supply exactly the unforgeable reference such an interface needs.

2.4 Reversible file operations

Journaling to enable undo is standard in filesystems and in version control. TIDY-UP predates this work with an append-only JSON Lines journal recording every move. The contribution here is the observation that a journal recording only *moves* is insufficient once an operation also *deletes* directories: replaying moves recreates directories that held files, by side effect, but silently loses directories that held nothing. Section 7.5 describes the additional record type this requires.

3 System overview

TIDY-UP is a Rust command-line tool, approximately 17,500 lines including tests, organized into layers with a strict dependency direction. Table 1 summarizes the modules, marking those introduced or substantially revised by this work.

An invariant governs the layering: *detection is pure, policy is impure*. The `safety` module answers the question “what kind of directory is this” without reading the environment, consulting a clock, or writing anything; the `commands::guard` module decides what to do about the answer, which requires a terminal and a user. This separation is what makes P3 (testability) achievable, and it recurs throughout: `plan`, `regroup` and `dedupe` compute reviewable data structures that `executor` later carries out.

4 System-directory detection

4.1 Interface

The classifier is a total function with no error case:

```
| pub fn classify(path: &Path, env: &Environment, facts: &Facts) -> Assessment;
```

Returning `Result` was considered and rejected. A classifier that can fail is one whose callers are tempted to write `unwrap_or(Safe)`, converting every transient error into a silent authorization. Instead, every input that could fail is hoisted into `Facts`, whose fields are all `Option`:

```
pub struct Facts {
    pub system_attribute: Option<bool>, // Windows FILE_ATTRIBUTE_SYSTEM
    pub reparse_point: Option<bool>, // Windows junction or volume mount
    pub mount_point: Option<bool>, // Unix: st_dev differs from parent
    pub owned_by_me: Option<bool>, // Unix ownership
    pub writable: Option<bool>, // empirical probe, Section 5
    pub fs_type: Option<String> // Linux /proc/self/mountinfo
}
```

Table 1: Module structure. Lines include in-file unit tests.

Layer	Module	Responsibility	Lines
classify	category	file type from extension	273
	rules	user ignore rules	239
	projects	detect code repositories	110
guard [†]	safety	is this a system directory	1,199
	safety_tests	rule-table corpus	1,227
	commands::guard	policy: warn, confirm, refuse	330
survive [†]	outcome	failure taxonomy and collection	531
discover	scan	directory traversal	505
decide	plan	organize plan	247
	dedupe	duplicate detection	533
	compare	multi-directory overlap	460
	regroup [†]	re-grouping plan	387
act	executor	perform a plan	250
	fsops	non-destructive primitives	416
remember	journal	append-only undo log	461
	restore	reverse a journal	315
observe [†]	obs	JSON Lines run record	608
report [‡]	api	JSON envelope, exit codes, plan files	1,016
delegate [‡]	mcp	Model Context Protocol server	1,085
permit [‡]	config	policy ceiling and default floor	807
review [‡]	commands::show	what one run did, and if it can be undone	476
present [†]	commands::session	menu-settable equivalents of the flags	371
present	cli, commands, ui	arguments, flows, terminal	1,700

[†] introduced for 0.2.0 [‡] introduced for 0.3.0 (Section 9)

A failed syscall yields `None`, which the classifier treats as *unknown* rather than as evidence in either direction. The corresponding test asserts that a `Facts::default()` — everything unknown — never produces a finding on its own.

4.2 Platform as a parameter

The single decision that most improves testability is making the platform an argument:

```
| pub enum Platform { Windows, MacOS, Linux, Other }
```

This forced a consequence that is easy to miss. Rust’s `std::path` parses paths for the *build* target: on Linux, `Path::new("C:\\Windows").components()` yields a single component, because the drive prefix is not recognized. Any classifier built on `std::path::Components` is therefore untestable across platforms by construction. We implement a platform-parameterized splitter:

```
| fn parts(path: &Path, platform: Platform) -> Parts; // (prefix, components)
```

`Parts` normalizes four things that are individually minor and collectively the source of most real bugs in path matching:

1. **Verbatim prefixes.** `\\?C:\Windows` and `C:\Windows` denote the same directory, as do

Table 2: Rule tables. **D** = Dangerous (refused), **C** = Caution (warn and confirm), **S** = carve-out suppressing broader rules. Carve-outs are evaluated first.

Platform	Level	Locations
Windows	D	%SystemRoot% (fallbacks %windir%, <drive>\Windows); %ProgramFiles%, (x86), %ProgramW6432%; %ProgramData%; <drive>\Users (the container); %USERPROFILE% itself; AppData; \$Recycle.Bin; System Volume Information; Recovery; Config.Msi; PerfLogs; \$WinREAgent; drive roots; UNC share roots
	C	%PUBLIC%; a bare reparse point
	S	anything strictly inside %TEMP%
macOS	D	/; /System (incl. /System/Volumes/*); /Library; /Applications; /usr; /bin; /sbin; /etc; /var; /private; /cores; /Network; /.vol; /Volumes and /Users as containers; \$HOME itself; \$HOME/Library
	C	/opt; /usr/local/*; \$HOME/Applications; \$HOME/.Trash; a bare mount point
	S	/Volumes/<disk>/*; /Users/<user>/*; /System/Volumes/Data/Users/<user>/*; /private/tmp; /tmp; %TEMP%
Linux	D	/; /bin; /sbin; /lib*; /usr; /etc; /boot; /dev; /proc; /sys; /run; /snap; /nix; /efi; /lost+found; /var; /home (container); /root; \$HOME itself; any pseudo-filesystem
	C	/opt; /srv; /usr/local/*; /media and /mnt as containers
	S	/tmp; /var/tmp; /media/<u>/<v>/*; /mnt/<x>/*; /run/media/<u>/<v>/*

\\?\UNC\srv\sh and \\srv\sh. An upstream function in TIDY-UP deliberately preserves the UNC verbatim form, so the classifier must fold both spellings.

2. **Case.** Windows and macOS compare case-insensitively by default; Linux does not. Components are case-folded only on the former.
3. **Trailing punctuation.** Windows silently strips trailing dots and spaces from names, so C:\Windows. resolves to C:\Windows and must match the same rule.
4. **Dot segments.** . is dropped and .. pops, so a path is judged by where it points.

Matching is then *component-wise*, never textual. The distinction is not pedantic: C:\Program Files Custom has C:\Program Files as a string prefix but is an entirely unrelated directory, and /usrlocal is not inside /usr. Both cases are asserted by name in the test suite.

Note also that upstream canonicalization closes an evasion path: because the target is resolved before classification, a symbolic link or NTFS junction pointing at /usr is judged as /usr. The original spelling is retained only for the message shown to the user.

4.3 Rule structure

Each platform has a table of entries and, for the Unix platforms, a list of *carve-outs* evaluated first. An entry is:

```
struct Entry {
    comps: &'static [&'static str], // components below the root
    exact: bool, // this directory only, or the subtree
    severity: Risk,
    label: &'static str, // human description
}
```

A carve-out either declares a subtree **Safe** with respect to the path rules or downgrades it to **Caution**. Table 2 gives the substantive content.

4.3.1 Windows: variables augment, never replace

Environment variables are read once, but the name-based rules are applied relative to *any* drive root regardless. A scrubbed environment, a service account, or a second Windows installation on D: therefore remains protected, while %SystemRoot% pointing at an unconventional location adds coverage rather than moving it. Two tests state this directly: one clears the variable map entirely and asserts the name rules still fire; the other sets SystemRoot=E:\Win and asserts that both E:\Win\System32 and C:\Windows are refused.

4.3.2 macOS: firmlinks and the data volume

Two macOS-specific hazards required carve-outs. /usr/local is writable and full of user-installed software, so the /usr rule is downgraded there rather than applied. More subtly, on APFS volumes a user’s home directory is reachable both as /Users/me and as /System/Volumes/Data/Users/me; canonicalization usually yields the short form, but not under all mount configurations. Without the second carve-out, a user’s own Downloads directory would be refused as part of /System.

System Integrity Protection has no public query interface. The rootless_check_trusted symbol is private SPI in libSystem and linking against it would be both unstable and inappropriate for a third-party tool. We treat the path table as a practical superset and rely on the writability probe (Section 5) as the backstop: SIP-protected paths reject writes even for root, so NotWritable fires, and that reason is deliberately non-overridable.

4.3.3 Linux: pseudo-filesystems

Rather than enumerate kernel-virtual mount points, we determine the filesystem type by parsing /proc/self/mountinfo. The parser is a pure function of the file contents and the target path, which permits fixture-based testing on any host:

```
| pub(crate) fn fs_type_from_mountinfo(text: &str, path: &Path) -> Option<String>;
```

The format requires care. Field 5 is the mount point, octal-escaped so that \040 denotes a space; a variable number of optional fields follows; a literal - separates them from the filesystem type. The longest mount point that is a component-wise prefix of the target wins, which resolves nested mounts correctly. Twenty-one pseudo-filesystem types are refused.

tmpfs is deliberately *absent* from that set. /tmp and /dev/shm are tmpfs, and /tmp is a perfectly reasonable directory to tidy. Including it would have been the more “complete” choice and the wrong one.

4.4 Composition

Algorithm 1 gives the composition. Three details are load-bearing.

The temporary-directory carve-out is load-bearing. On two of three platforms the system temporary directory lies inside a protected subtree: Windows places it under %LOCALAPPDATA%\Temp, and macOS canonicalizes it to /private/var/folders/..., matching both /private and /var. Without the carve-out the guard would refuse the system temporary directory — a legitimate target — and, because the project’s entire pre-existing test suite builds its fixtures with tempfile, would have broken every test on first commit. The dependency between a safety rule and the test infrastructure was not anticipated during design and was discovered only when enumerating the rule table.

Algorithm 1: CLASSIFY

Input: path p , environment e , facts f
Output: assessment with risk level and ordered reasons

- 1 $P \leftarrow \text{parts}(p, e.\text{platform})$
- 2 $R \leftarrow \emptyset$
- 3 $\text{inTemp} \leftarrow P$ strictly inside $\text{parts}(e.\text{temp})$
- 4 **if** P has a prefix and no components **then**
- 5 $R \leftarrow R \cup \{\text{UncShareRoot if UNC else FilesystemRoot}\}$
- 6 **end**
- 7 **if** $P = \text{parts}(e.\text{home})$ **then**
- 8 $R \leftarrow R \cup \{\text{HomeRoot}\}$
- 9 **end**
- 10 **if** $\neg \text{inTemp}$ **then** $R \leftarrow R \cup \text{pathRules}(P, e)$
- 11 **if** $f.\text{fs_type} \in \text{PSEUDO}$ **then**
- 12 $R \leftarrow R \cup \{\text{PseudoFilesystem}\}$
- 13 **end**
- 14 **if** $f.\text{system_attribute} = \text{Some}(\top)$ **then** $R \leftarrow R \cup \{\text{SystemAttribute}\}$
- 15 **if** $f.\text{writable} = \text{Some}(\perp)$ **then** $R \leftarrow R \cup \{\text{NotWritable}\}$
- 16 **if** $f.\text{owned_by_me} = \text{Some}(\perp)$ **then** $R \leftarrow R \cup \{\text{NotOwned}\}$
- 17 **if** $(f.\text{mount_point} \vee f.\text{reparse_point}) \wedge R = \emptyset$ **then**
- 18 $R \leftarrow \{\text{MountPoint}\}$ // only when nothing stronger fired
- 19 **end**
- 20 sort R by severity, descending
- 21 remove $r \in R$ whose explanation duplicates an earlier one
- 22 **return** $(\max_{r \in R} \text{severity}(r), R)$, or **Safe** if $R = \emptyset$

Mount points are suppressed when anything stronger fires. Every external drive is a mount point. Reporting it alongside “this is /usr” adds noise without information, so the reason is emitted only when it is the sole finding, and then only at **Caution**.

Reasons are deduplicated by rendered explanation. C:\Windows matches the %SystemRoot% rule, the %windir% rule and the name rule simultaneously. The first implementation printed three near-identical lines. Deduplicating structurally is insufficient, because the three findings differ in their `matched` field; deduplicating by the sentence the user actually reads is both simpler and correct. This was found by running the tool against the real C:\Windows, not by any test, and a regression test was added afterwards.

4.5 Deliberately weakened rules

Two candidate rules were weakened or removed on precision grounds (P2).

“*Outside the home directory*” was removed entirely. It is a plausible heuristic and would have been catastrophic in practice: D:\Downloads, /mnt/photos and /Volumes/Backup are the *normal* targets for this class of tool. A rule that fires on the normal case teaches users to pass the override flag reflexively, after which the guard protects nobody.

“*Shallow path*” (a directory one level below a filesystem root) was retained only in conjunction: it fires only when the directory is additionally not under the user’s home and demonstrably not owned by the current user, and even then only at **Caution**. /data, /srv2 and D:\Media are ordinary directories that a naive depth rule would flag.

The general principle we extracted: *composition, not any single weak signal*. A directory that is shallow *and* unowned *and* unwritable *and* a mount point is almost certainly system-managed;

Table 3: Guard policy. `-yes` answers a confirmation; it never substitutes for the override flag.

Risk	Reporting command	Destructive command	With <code>-allow-system-folder</code>
Safe	silent	silent	silent
Caution	warn	warn, then confirm (default <i>no</i>)	same
Dangerous	warn, proceeds	refused , non-zero exit, names the flag	warn, then confirm (default <i>no</i>)
NotWritable	warn, proceeds	refused	still refused

any one of those alone is not.

4.6 Policy

Policy is separated from detection and lives in `commands::guard`. Table 3 states it completely.

Four aspects warrant comment.

-yes is not an override. This is the property the evaluation treats as flagship. Automated invocations that pass `-yes` continue to work on ordinary directories and begin failing loudly on system ones. The resulting breakage of existing scripts is intended, and is the mechanism by which the guard reaches unattended use at all.

The override reaches the question, not past it. With `-allow-system-folder` the user is still asked, the default answer is *no*, and with no terminal attached the run stops. Obtaining a destructive outcome on a flagged directory requires the flag *and* an affirmative answer.

A permission wall is not overridable. No command-line flag grants permission that the operating system refused, and pretending otherwise would produce a run that fails item by item after the user has already been warned. The refusal message says explicitly that the flag will not help.

Reporting commands never block. `analyze` and `history` warn and continue. Analyzing an entire drive is an advertised use of the tool; blocking it would have been a regression, and an existing test covers it.

Two further points of mechanism. First, the guard runs at the single choke point where a command turns a user-supplied path into a validated root, and *not* inside the library function that performs that resolution: that function is pure, returns a library error type, cannot prompt, and is called by multi-directory validation paths where a policy decision would be wrong. Second, `distribute` guards its destinations as well as its sources, since `-to C:\Windows` is as damaging as `-from`; a destination that does not yet exist is judged by the nearest existing ancestor, which is where it would be created.

Finally, `-include-hidden` escalates Caution to Dangerous. Bringing dotfiles into scope widens the blast radius into exactly the territory where configuration and system state live; a Safe directory remains Safe, because hidden files in a user's own downloads directory are not a hazard.

4.7 Reaching the override without a command line

An escape hatch that cannot be reached is not an escape hatch. Table 3 promises that a determined user can proceed (P4), but the promise is carried entirely by a command-line flag, and the tool is also usable by running it with no arguments, which starts a menu. Someone who launched it by double-clicking has nowhere to put a flag. For them, P4 did not hold: not only was the override unreachable, so were preview mode, hidden-file inclusion and every ignore rule, because the menu constructed each command with default arguments.

This is easy to miss when the guard is designed, tested and demonstrated from a shell. The fix is a settings screen holding the same options for the duration of the session, with whatever is not at its default printed above the menu so the next action is never a surprise. Nothing is persisted: the tool writes no configuration file outside the folders it processes, and that property was worth more than remembering a preference between runs.

Two details preserve the policy rather than route around it. Enabling the override asks for confirmation, and the guard still asks again before acting, so it remains two deliberate steps rather than one sticky setting. And “skip confirmations”, the menu equivalent of `-yes`, is deliberately absent: in an interactive session the confirmation *is* the review step, and there is no script waiting on it.

The general point is that a safety mechanism has to be evaluated against every way the program can be started, not only the one its author uses.

5 Permission determination

5.1 Empirical probing

Given the disagreements catalogued in Section 2, we determine writability by attempting it:

```
fn probe_writable(dir: &Path) -> Option<bool> {
    let name = format!(".tidy-up-write-test-{}-{}-{}", process::id(),
        match fs::File::create_new(dir.join(&name)) {
            Ok(f) => { drop(f); let _ = fs::remove_file(dir.join(&name)); Some(true) }
            Err(e) if e.kind() == PermissionDenied
                || e.kind() == ReadOnlyFilesystem => Some(false),
            Err(_) => None, // out of space, directory vanished: unknown, not "no"
        }
    }
}
```

Three properties matter. The name is unique per process, per nanosecond and per monotonic counter, so concurrent runs cannot collide. `create_new` fails rather than truncating if the name somehow exists. Most importantly, only `PermissionDenied` and `ReadOnlyFilesystem` produce `Some(false)`: every other error yields `None`, because an unknown must never be reported as an accusation. A test asserts exactly this for a directory that does not exist.

The probe is *scoped*. It runs only for commands about to write, and never under `-dry-run`. A reporting command and a dry run both promise to leave the disk untouched, and that promise outranks the diagnostic value of knowing writability for an operation that was not going to write. Two tests assert that the directory is still empty after a read-only assessment and after a dry-run assessment.

5.2 Ownership, and a dependency declined

Unix ownership is available cheaply, but the standard library exposes no way to read the current *effective* user ID, so comparison requires `libc` or `rustix`. We shipped without ownership, retaining `Facts::owned_by_me: Option<bool>` so it can be supplied later without changing the classifier or any test. The justification is that the writability probe already answers the question that determines the outcome, that `NotOwned` is only ever `Caution`, and that the project’s zero-dependency constraint (Section 10.7) is a stated property of the artifact. Should ownership prove necessary, `rustix` is preferred over `libc` because it is implemented in pure Rust on `linux-raw` and preserves the dependency-free static musl build.

5.3 Reporting refusals distinctly

A permission failure previously surfaced as `/srv/data: Permission denied (os error 13)` or, below the root, as the single word “unreadable”. We separated it throughout: a new `Error::Denied` variant renders as `cannot read <path>: permission denied` and permits the command layer to add an actionable hint, and `SkipReason` gained a `Denied` variant distinct from `Unreadable`. The difference is the difference between a user concluding “TIDY-UP is broken” and “I do not own that directory”.

5.4 Hidden and system-owned files

Three defects in hidden-file handling were found and corrected.

1. **macOS UF_HIDDEN.** Files hidden with `chflags hidden` carry a flag in `st_flags` that no leading-dot check detects. `std::os::macos::fs::MetadataExt::st_flags` is stable, so this cost nothing but the recognition that it was missing.
2. **Windows *hidden* conflated with *system*.** The previous implementation tested the mask `0x2 | 0x4` as a unit, so `-include-hidden` also un-skipped every file Windows marks as its own. Asking to see dotfiles is not asking to move `pagefile.sys`. The attributes are now tested separately, a `SkipReason::SystemAttribute` reports the second, and it is *not* governed by `-include-hidden`.
3. **Escalation.** As noted above, including hidden items raises a `Caution` verdict to `Dangerous`.

6 Survival architecture

6.1 Principle

A bulk operation over thousands of files will encounter a locked file, an unreadable directory, or a name the filesystem rejects. Aborting costs the user the entire remaining run; worse, it leaves the operation half-complete in a way the user did not choose. The principle adopted is:

Fail the run only when continuing would do something the user did not ask for, or would lose the ability to undo it. Everything else is a skip: classify it, record it, step over it, and report it at the end.

6.2 Failure taxonomy

A single type carries every failure, and one function classifies every I/O error, so that counting is consistent across modules:

```
pub struct Problem { path: PathBuf, op: Op, cause: Cause, message: String }

pub enum Cause { Denied, NotFound, InUse, Exists, CrossesDevices,
                NoSpace, Invalid, Panic, Other }
```

`Cause::of` maps `io::ErrorKind` to a cause in one place. Two mappings are worth noting. `ReadOnlyFilesystem` maps to `Denied`, because from the user’s point of view it is a permission problem with the same remedy. Windows reports a file held open by another process as `ERROR_SHARING_VIOLATION` (32) or `ERROR_LOCK_VIOLATION` (33), neither of which has a stable `io::ErrorKind`; these are recognized by raw code and mapped to `InUse`, or every locked file would be reported as “other”.

Two combinators express the principle directly:

```
pub fn try_or_skip<T>(&mut self, path: &Path, op: Op,
                    f: impl FnOnce() -> Result<T>) -> Option<T>;
pub fn catch_or_skip<T>(&mut self, path: &Path, op: Op,
                       f: impl FnOnce() -> Result<T> + UnwindSafe) -> Option<T>;
```

The second additionally contains panics via `catch_unwind`, converting a bug in the processing of one file into a `Cause::Panic` problem rather than the loss of the entire run. A test panics inside the closure and asserts both that the problem is recorded with its message and that the collection remains usable afterwards.

6.3 Converting the traversal

The directory walk previously propagated `Result` through its recursion, so a failure anywhere in a large tree aborted the scan. It is now infallible below the root: every failure becomes a `Skipped` entry with a classified reason. Only the root itself can fail the command, which is correct, because a root that cannot be listed offers nothing to do.

A test constructs a tree with one subdirectory at mode `0o000` and asserts both that every other file is still returned and that the failed directory is named exactly once with reason `Denied`. It self-disables when running as root, for whom nothing is unreadable.

6.4 The one fatal exception

Journal write failure remains fatal, and the in-flight move is rolled back. This is the deliberate exception to the principle: a change that cannot be recorded cannot be undone, and silently continuing would strand files with no way back. Stating the exception explicitly, and having exactly one, is what makes the rest of the policy legible.

6.5 Reporting

Reporting is grouped by cause with an actionable hint per group, then individual paths, capped unless `-verbose` is given:

```
! 23 items could not be processed:
  Permission denied 18 (pick a folder you own, or run as an administrator)
  In use by another program 4 (close the programs holding these files and run again)
  No longer there 1 (these were deleted or renamed while tidy-up was running)
  Downloads/locked.pdf: ...
  ... and 13 more (use --verbose to list them all)
```

Twenty identical lines reading “permission denied” inform nobody; one line reading “18 permission denied, pick a folder you own” is actionable. A run that skipped items still exits zero, because it performed the work it could; `-strict` makes it exit non-zero for callers that need to know.

7 Reversible re-grouping

7.1 Motivation and interface

The pre-existing `organize` command sorts loose files into category directories and deliberately refuses to descend into its own output, scanning one level deep and skipping known category names. That is correct for a downloads directory and useless for a directory somebody organized badly years ago, which is the common real case.

`reorganize` inverts both choices. It descends without limit, including into directories a previous run created, re-files every file under an ordered list of grouping keys, and deletes the directories it empties.

Table 4: Grouping keys. Every key is total: no file is left unplaced.

Key	Example directory	Derivation
type	Images, 3D Models	shared with <code>organize</code> , so the two commands agree
ext	PDF, No extension	upper-cased extension; the “document type” grouping
year	2024, Unknown date	modification time
month	03-March	numeric prefix so directories sort in calendar order
day	2024-03-17	modification time
size	Small (under 10 MiB)	five bands, powers of 1024
alpha	A, 0-9, Other	first character, upper-cased

```

tidy-up reorganize <path> --by year,type # 2024/Images/photo.jpg
tidy-up reorganize <path> --by type,ext # Images/PNG/photo.png
tidy-up reorganize <path> --by alpha

```

Table 4 lists the keys. Each becomes one level of nesting, in the order given.

Totality is enforced by test: a corpus of awkward inputs — no extension, no timestamp, a leading digit, an accented initial (`étude.mp3`), a non-Latin initial (`nihongo.txt` ^(non-Latin)), and the name ... — is passed through every key, and each is asserted to produce a non-empty, already-sanitized name.

7.2 Name sanitization

Directory names derived from file contents can contain anything. Sanitization replaces the nine characters Windows rejects and all control characters, trims trailing dots and spaces, falls back to `Other` if nothing survives, and appends an underscore to the 22 reserved device names. The last case is not hypothetical: `driver.con` is an ordinary file name, and grouping by extension would otherwise attempt to create a directory named `CON`, which Windows will not create at any path.

7.3 Idempotence

`organize` obtains idempotence from its skip list. `reorganize` has no skip list and must compute it: for each file, the destination is derived, and if it equals the current path no move is recorded. A move from a path to itself would otherwise churn the journal on every run and never converge.

The property is asserted twice, at two levels. A unit test feeds the destinations of one plan back in as the input of a second and asserts the second plan is empty. An integration test runs the compiled binary twice over a fixture and asserts that the second run reports nothing to do and that the file and directory listings are byte-identical.

7.4 Computing emptied directories

Algorithm 2 computes which directories the run will leave empty. The subtlety is that emptiness is transitive upward — a directory containing only directories that become empty is itself empty — while any retained item disqualifies a directory *and all of its ancestors*.

Items are *retained* when they are skipped by an ignore rule, are unreadable, are symbolic links, or are code project directories that policy keeps in place. The tool’s own `.tidy-up` state directory is excluded unconditionally: it holds the undo journals, and a run that swept away its own means of reversal would be self-defeating.

Two safety layers back the computation. Directories are removed deepest first, and removal uses `remove_dir`, which succeeds only on a genuinely empty directory. An error in Algorithm 2 therefore produces a directory that is not removed, never one that is removed with contents. Consistent with Section 6, a removal that fails is recorded as a problem and the run continues.

Algorithm 2: EMPTIEDIRECTORIES**Input:** root r ; V = parents of moved items; K = ancestors of retained items**Output:** directories to remove, deepest first

```

1  $C \leftarrow \emptyset$ 
2 foreach  $d \in V$  do
3   while  $d \neq r$  and  $d$  is under  $r$  do
4      $C \leftarrow C \cup \{d\}$ ;  $d \leftarrow \text{parent}(d)$ 
5   end
6 end
7  $C \leftarrow C \setminus K$  // anything holding a survivor, and its ancestors
8  $C \leftarrow C \setminus \{d : d \text{ is or is under } r/.tidy\text{-up}\}$ 
9 sort  $C$  by component count, descending
10 return  $C$ 

```

7.5 Undo

Reversing a re-grouping requires more than replaying moves. Files whose parent directories vanished are restored correctly by side effect, since the move primitive creates destination parents. Directories that ended up holding *nothing* have no move to recreate them and would be silently lost.

The journal therefore gained one record type:

```
| DirRemoved { path: PathBuf } // relative to the root, like every other record
```

written after each successful removal — after, so that a removal which did not happen is never recorded. Restore replays records in reverse, which places directory recreation before the moves that repopulate the subtree.

The property is asserted end to end. An integration test constructs a deliberately badly organized tree — nested category directories, a `_Duplicates` directory, a code project, a dotfile, and an already-empty directory — reorganizes it by `year,type`, then restores, and asserts that both the complete file map (relative path to contents) and the complete directory listing equal their original values. The empty directory is in the fixture specifically because it is the case the journal could not previously express.

7.6 Time zones: a stated limitation

Date grouping requires civil dates, and the Rust standard library has no time-zone interface. Obtaining the local UTC offset portably would require a TZif parser plus `/etc/localtime` on Unix and a registry read on Windows — in practice, a dependency.

Rather than be correct on one platform and silently wrong on another, dates are UTC, and an environment variable `TIDY_UP_UTC_OFFSET` accepting `+03:00`, `-0530`, `+7` or `Z` applies an offset. The consequence is real and documented: with no offset set, a file modified at 01:00 local time in UTC+3 on 1 January is grouped under the previous year. A test states exactly this case in both configurations. We consider a documented, correctable limitation preferable to an undocumented platform-dependent one, and note that the effect is confined to a small fraction of files near boundaries and is fully reversible.

8 Observability

8.1 What the journal already provides

An important negative design decision: the run log is *not* a per-move log, because one already exists. The undo journal records every move as it happens, unbuffered, so that a crash leaves a usable record. Duplicating it would double the writes to say the same thing twice.

What a journal of moves cannot record is everything that did *not* happen: the safety verdict, what was skipped and why, what failed, and how long each stage took. That is the gap the run log fills, and the two artifacts join on a shared identifier.

8.2 Format and policy

Events are buffered and written once, at the end of the run, to `<root>/tidy-up/logs/<run-id>.jsonl`, beside the journal. Buffering is acceptable precisely because the journal is the crash-safe record: a run log missing after a crash costs a diagnosis, not data. A complete log from a real run:

```
{
  "type": "run",
  "command": "reorganize",
  "version": "0.1.1",
  "os": "windows",
  "format": 1,
  ...
}
{
  "type": "safety",
  "path": "...\\logdemo",
  "risk": "safe",
  "reasons": [],
  "allowed": false
}
{
  "type": "scan",
  "root": "...",
  "files": 2,
  "bytes": 3,
  "projects": 0,
  "skipped": {},
  "ms": 0
}
{
  "type": "plan",
  "operation": "reorganize",
  "moves": 2,
  "bytes": 3
}
{
  "type": "execute",
  "journal_id": "20260928-204156",
  "moved": 2,
  "bytes": 3,
  "dirs_removed": 2,
  "failed": 0,
  "ms": 0
}
{
  "type": "end",
  "status": "ok",
  "ms": 4,
  "metrics": {
    "roots": 1,
    "files_scanned": 2,
    "bytes_scanned": 3,
    "skipped": 0,
    "planned_moves": 2,
    "moved": 2,
    "bytes_moved": 3,
    "dirs_removed": 2,
    "failed": 0,
    "denied": 0
  }
}
```

Events are a closed Rust enum serialized with an internal tag, which makes the log a type-checked, versioned contract rather than a collection of stringly-typed fields that a typo can silently rename — the same property the journal format already has. `status` distinguishes `ok`, `error`, `cancelled` and `blocked`, the last recording a guard refusal.

Logging is off by default, enabled by `-log` or `TIDY_UP_LOG`, preserving the project's documented promise to write nothing the user did not ask for. Logs are pruned to the newest twenty per directory; journals, which are undo data, are never touched by pruning, and a test asserts this by placing a non-log file in the directory and verifying it survives. A log that cannot be written degrades silently: a diagnostic must never change a run's outcome or exit code, and a test asserts that a run whose log directory path is occupied by a regular file still succeeds.

8.3 A conceded trade-off

The sink is process-level rather than threaded through the call graph. A command-line tool performs exactly one run per process, and threading a handle through nine commands to record six events would have cost more in noise and merge risk than it returned. The cost is that tests of the sink exercise it directly or through the compiled binary rather than in parallel against isolated instances. We consider this the right trade at this scale and would revisit it if the library were embedded.

9 Delegating to an autonomous agent

The mechanisms so far protect a person from a mistake. This section is about a different caller: a program that decides what to do without being told each time. The distinction matters because an agent acts in bulk, acts quickly, and acts on text it was given, which may not have come from the person it is working for.

Table 5: Exit codes. 3 is the one that did not exist before.

Code	Meaning
0	Success, including “nothing to do”, a cancelled confirmation, and a run that skipped items it could not process
1	Something went wrong and stopped the run
2	The arguments did not make sense
3	Refused deliberately: the system-folder guard, a permission wall, or policy
4	Finished, but items were skipped and <code>-strict</code> was given

Everything here is built on a property the earlier work already had, and which we think is the precondition for delegating filesystem mutation at all: *every action can be reviewed before it happens and reversed after*. A `Plan` is pure data (Section 3), and every change is journaled (Section 7.5). Without both, no amount of interface design makes this safe; with both, the remaining question is what the agent is allowed to express.

9.1 A machine interface

Every command accepts `-json` and prints exactly one object on standard output, whatever happened:

```
| {"tidy_up":"0.3.0","format":1,"command":"organize","status":"ok","outcome":{...}}
```

The human and machine renderings are two views of one value, not two implementations: a command returns an `Outcome` and the caller decides how to render it. Keeping one model is what stops the two drifting.

Making that reliable required a single choke point rather than a guard at each call site. A `out!` macro replaces every prose `println!` in the crate, and progress bars are suppressed. Guarding each site individually would have worked on the day and failed silently the next time somebody added a print, which is the failure mode worth designing against.

Exit codes distinguish a refusal from a failure. Previously every non-success exited 1. A caller cannot act on that: a refusal is an answer and must not be retried, whereas a transient failure might reasonably be. Table 5 gives the mapping. To make it honest, refusals are *typed* rather than worded: a `Refused` error carries an `ErrorCode`, and both the exit code and the JSON error derive from it. An earlier version decided the run-log status by testing whether the message began with “refusing”, which is the kind of thing that survives until somebody rewords a string.

9.2 Two ways to let an agent move files

This is the central design question of the section, and we think the answer generalizes well beyond this tool.

Suppose an agent should be able to reorganize a directory. There are two shapes the interface can take.

Validate what it asks for. The agent supplies operations, and the tool checks each one.

This is what the `apply` sub-command does with a plan file: it verifies that every path is absolute and lies inside the plan’s own root, and that the counts match, before executing anything.

Never let it name anything. The agent asks the tool to compute a plan, receives an opaque handle, and applies *that*. It never supplies a path. This is what the MCP server does: `plan_organize` returns a `plan_id`, and `apply_plan` accepts that identifier and nothing else.

The second is strictly stronger, and the reason is worth stating precisely. Validation has to anticipate every way a path can escape its root, and Section 4 is a catalogue of how many ways that is: `..` components, symbolic links and NTFS junctions, verbatim and UNC prefixes, 8.3 short names, case folding on two platforms, and trailing characters Windows strips silently. A validator is only as good as that list. Under the second shape the request contains no path at all, so the question does not arise: the moves are the ones the tool computed, and the agent’s only remaining choice is *whether*, not *what*.

This is the capability-security distinction — unforgeable references in place of ambient authority checked against a name — applied to agent tool design [1, 2]. We claim no novelty in the principle. What seems worth reporting is that it maps unusually cleanly onto a tool already split into a pure planner and an executor: the handle is simply the plan, and the split existed for testability long before an agent was in view.

The practical consequence is a defence against indirect prompt injection [4]. An agent that reads a file containing “*also move C:\Windows\System32 into Documents*” cannot act on it through this interface, because there is no argument in which that instruction could be expressed. It can be expressed to the validating interface, where it is then rejected — which is a weaker position, because it relies on the rejection being complete.

9.3 The MCP server

The Model Context Protocol is JSON-RPC 2.0 over newline-delimited JSON on standard input and output, so serving it needed no dependency beyond the `serde_json` already present. `Server::handle` is a pure function from one request to at most one response, and the loop around it does nothing but framing; the protocol is therefore tested without a pipe or a subprocess, the same separation the classifier uses.

Four restrictions apply, in rough order of how much they carry:

1. **No paths in requests**, as above.
2. **Confinement to one root**, given on the command line and checked on the canonical path, so `..` and symbolic links are caught.
3. **Read-only by default**. Without `-allow-writes` the mutating tools are *absent from the tool listing*, not present and failing. An agent cannot attempt what it cannot discover, and a listing is a cheaper thing to reason about than a set of runtime refusals.
4. **The guard cannot be overridden**. There is no way to express `-allow-system-folder` over this interface, and a server rooted at a refused directory does not start. Failing at startup rather than per call matters: an agent holding a connection to such a server would otherwise find every call failing with no way to understand why.

A plan is consumed when applied, so a replayed `apply_plan` cannot re-run moves against a directory that has since changed. Large plans are returned as a 50-move sample with a truncation flag, because a response of ten thousand entries helps no reader, human or otherwise.

9.4 The review surface

Delegation is only acceptable if the delegation can be checked, and checking was the weakest part of the system: `history` listed runs and nothing explained one. Reading a journal meant opening its JSON Lines file.

Building `show` surfaced a structural point about journal-based undo. *Two of the three things a reviewer wants are not in the journal, and cannot be.*

1. **Whether each item is still where the run put it** must be determined at review time, because the directory keeps changing after the run. This is the question that actually matters, since it is the difference between an undo that is clean and one that reports items as missing. The report marks each moved-on item.
2. **What the run left alone** can never be in a journal, because a journal records changes. It comes from the optional run log (Section 8), and its absence is reported as silence rather than as an error, since most runs are not logged.
3. Timing, which the journal does have.

So `show` is not a journal pretty-printer; it is a journal, plus a live filesystem check, plus a separate log. We suspect this generalizes to any journal-based undo facility: the journal is a record of a transition, and a reviewer is asking about a state.

9.5 Configuration: a ceiling and a floor

An agent-facing tool needs configuration for a different reason than a human-facing one. A person wants to avoid retyping flags; an agent does not care. What an installation needs is a statement of what it *permits*, which the caller cannot override.

The file therefore holds two things with opposite semantics. A **policy** is a ceiling: permitted roots, whether writes are allowed at all, whether deletion is, whether the override flag exists, and a cap on scan depth. Flags cannot raise it. **Defaults** are a floor: ignore rules, depth, grouping keys. Any flag overrides them.

Discovery is deliberately asymmetric. A policy is read only from a path the operator named, via `-config` or an environment variable, and is never discovered. Defaults may additionally come from a `.tidy-up.json` inside the directory being processed, which is convenient and travels with the directory; a policy found there is discarded.

The asymmetry is the security property. If a policy could be discovered from the directory being processed, then unpacking an archive and tidying it would let the archive decide what may be done to it — a confused-deputy arrangement [3] in which the tool’s authority is directed by the data it was pointed at. The rule we extracted is short enough to apply elsewhere: *auto-discovered configuration may lower the bar for convenience; it may never raise it for permission.*

Two smaller decisions follow the same instinct. Passing a flag that policy forbids is an *error*, not a silent no: the caller believes they have authorized something, and learning otherwise afterwards is worse than being told. And an unrecognized field is rejected rather than ignored, because `allow_system_folder` for `allow_system_folders` would otherwise leave an installation believing it was locked down when it was not.

10 Evaluation

10.1 Method

Evaluation is by automated test, at three levels: pure unit tests of the classifier and planners, host-dependent unit tests of the probes, and black-box integration tests that execute the compiled binary. All figures below are from a full `cargo test` run on Windows 11 with Rust 1.95.0. Table 6 gives the counts.

Table 6: Test suite. 455 tests, zero failures, zero clippy warnings at `-all-targets`, on both stable Rust and the declared minimum of 1.85.

Suite	Tests	Scope
unit (in-crate)	331	classifier corpus, probes, planners, taxonomy, log and JSON formats, MCP protocol, policy
tests/cli	30	pre-existing command-line behaviour (regression)
tests/workflow	23	scan → plan → execute → journal → restore
tests/reorganize	11	re-grouping, idempotence, undo round trip
tests/safety	11	guard policy through the compiled binary
tests/api	10	envelope shape, exit codes, propose/apply, plan trust boundary
tests/config	10	policy ceiling, defaults floor, the folder-config attack
tests/show	8	review surface, clean versus partial undo
tests/logging	7	run log content, joins, degradation, <code>-strict</code>
tests/cross_volume	5	cross-partition moves (environment-gated)
tests/scripts	3	installer scripts
doc-tests	6	every documented example compiles and runs
Total	455	

10.2 Cross-platform rule validation

Because Platform is a parameter, the full rule corpus runs on every host. The table-driven suite covers Windows dangerous locations and safe locations, scrubbed and redirected environment variables, case and separator variants, macOS dangerous locations and all five carve-outs, and Linux dangerous locations and carve-outs, each case carrying a one-line justification that is printed on failure. This is the concrete payoff of P3: the macOS `/System/Volumes/Data` carve-out and the Linux `/run/media` carve-out — the two most consequential and most easily forgotten — are verified on a Windows development machine, where neither path exists.

10.3 Precision: the negative corpus

P2 is tested directly by `ordinary_folders_are_never_flagged`, which asserts Safe for 24 ordinary directories spanning all three platforms: `C:\Users\me\Downloads`, `D:\Media\Movies`, `\\nas\media\Photos`, `/Users/me/Documents/Work`, `/Volumes/Backup/Photos`, `/home/me/Pictures/2024`, `/mnt/storage`, `/media/me/Elements`, `/data/archive`, `/tmp/scratch` and others. The test exists as a regression net: a future rule that increases coverage at the expense of precision fails it, and the failure names the directory that regressed.

10.4 Guard behaviour

Ten integration tests drive the compiled binary with no terminal attached, the condition that matters for scripts and CI. The most significant assert that `organize <home> -yes` exits non-zero, names the override flag, and leaves the home directory listing unchanged; that the override alone still cannot run unattended; that `-dry-run` on a flagged directory warns, proceeds and creates no state; that reporting commands are never blocked; that `distribute` guards destinations; and that *every* destructive command refuses the home directory — the last written as a loop over the command set so that a newly added command without a guard fails the suite. A complementary test asserts that reporting commands are the exception, so the first cannot be satisfied by blocking everything.

Manual verification on the development machine confirmed the intended behaviour against real system directories:

```
| $ tidy-up organize 'C:\Windows' --yes
```

Table 7: Dependency footprint before this work and at release 0.3.0.

	Before	After
Direct dependencies	10	10
Unique transitive crates	54	54
Stripped release binary	2.19 MiB	2.68 MiB
New dependencies added	0	
Lines added (implementation + tests)	10,265	

```
Careful
! C:\Windows is a folder the system manages.
  It holds the Windows installation, which other programs expect to find in place.
  tidy-up cannot write here, so nothing could be moved.
error: refusing to change C:\Windows: tidy-up cannot write there, and
      --allow-system-folder cannot grant permission the system refused
$ echo $?
1
```

Note that `C:\Windows` is refused on two independent grounds — the name rule and the writability probe — and that the message correctly reports the override as unhelpful, since the binding constraint is permission rather than policy.

10.5 Reversibility

The re-grouping round trip is asserted as equality of two complete observations of the tree, before and after, excluding only the tool’s own state directory: a map from relative path to file contents, and a sorted list of relative directory paths. Testing directory structure separately from file contents is what gives the test power over the `DirRemoved` record; a files-only comparison would pass even if every empty directory were lost.

10.6 The delegation interface

The MCP server is exercised through `Server::handle` directly, since it is a pure function of a request, with 18 tests covering the handshake and version negotiation, notifications receiving no reply, tool errors arriving as results rather than protocol failures, and the full plan/apply/review/undo cycle.

Four of them are the safety argument of Section 9.2 stated as assertions. One confirms that `apply_plan` advertises exactly one input property, `plan_id`, so the interface admits no path. One confirms that an invented handle is refused. One confirms a plan cannot be applied twice. The fourth attempts four escapes from the server root — `..`, a trailing-separator variant, an absolute path elsewhere, and on Unix a symbolic link planted inside the root pointing out of it — and requires all four to be refused.

The plan file interface, being the weaker of the two shapes, is tested adversarially in kind: a proposed plan is rewritten so that one destination is `C:\Windows`, and the test requires both that the run is refused and that the target does not come into existence.

For configuration, the corresponding test writes a `.tidy-up.json` into a directory that the operator’s policy does *not* permit, claiming in it that the directory is permitted and that system folders are allowed, then requires the run to be refused anyway. That is the confused-deputy scenario of Section 9.5 reduced to a test case.

10.7 Dependency footprint

Zero new dependencies was a hard constraint, and it shaped four decisions that would otherwise have gone differently:

1. **Logging** is hand-rolled on the already-present `serde`, not tracing with `tracing-subscriber`. The ecosystem option would have added roughly a dozen transitive crates, dominated by `regex-automata` via `EnvFilter`, for dynamic filtering and asynchronous span propagation that a single-threaded tool with one process-wide span does not use. The closed event enum is, additionally, a better contract than stringly-typed fields.
2. **Ownership checks** were dropped rather than take `libc` or `rustix` (Section 5).
3. **Local time** was declined rather than take a date crate (Section 7).
4. **Platform attributes** use stable `std::os::*::fs::MetadataExt` interfaces — `file_attributes` on Windows, `st_flags` on macOS, `dev` on Unix — rather than `windows-sys`.

We report items 2 and 3 as genuine reductions in capability accepted in exchange for a stated property of the artifact, not as free wins.

The binary grew by roughly half a megabyte across both releases, which is the cost of the code itself rather than of anything linked in: the dependency graph is the same 54 crates it was before any of this started. For comparison, the single rejected dependency of item 1 was estimated at a similar size on its own.

10.8 What continuous integration caught

Three classes of defect survived a green local run on Windows and were caught only by the cross-platform matrix. They are reported because each is a direct consequence of a design decision described above.

First, **the temporary-directory carve-out was broken on two of three platforms**, exactly the mechanism Section 4 identifies as load bearing. The environment recorded the canonicalized temporary directory, but a caller that has not canonicalized its own path supplies a different spelling of the same place: `tempfile` returns `/var/folders/...` on macOS where the canonical form is `/private/var/folders/...`, and an 8.3 short name on Windows. The two never compared equal, so every temporary directory was classified *Dangerous*. The fix records the temporary directory under every spelling and, on macOS, folds `/private/var`, `/private/etc` and `/private/tmp` onto their short names, since a verdict must not depend on how the caller typed the path. Three tests now pin this, written against the pure classifier so they run on any host. This is a pointed illustration of the limits of P3: making the *rules* testable everywhere does not make the *environment* so.

Second, **the new code did not compile on the declared minimum supported Rust version**. Ten sites used `let` chains, stabilized in 1.88, and one used `ErrorKind::InvalidFilename`, still feature-gated at 1.85. Neither is visible to a developer on current stable. A declared MSRV is a compatibility claim like any other and needs a job that checks it.

Third, **a defect from reading the clock twice**. The run log and the undo journal each mint an identifier from a separate call to `compact_id(now_secs())`, on the assumption they would agree. A run that crosses a one-second boundary between starting and opening its journal ends up with two, and `show` was locating the log by the journal's name. It therefore reported nothing about what the run left alone — indistinguishable, to a reader, from a run that was never logged. Only the musl job happened to be unlucky in the second that mattered. The fix matches on the identifier the log already records, and the general lesson is that two clock reads are two values: if one artifact must refer to another, it should say so rather than be named after it.

Fourth, **two integration tests encoded assumptions about the developer’s machine**: one required the home directory to contain loose files, which a fresh CI runner’s does not, and one compared a directory listing that a parallel test was concurrently modifying.

The second of those turned out to be a design defect rather than a flaky test. The two listings differed by a file named `.tidy-up-write-test-...`: the guard gathered every fact before classifying, so the writability probe of Section 5 created a file *inside the very directory it was about to refuse*. Harmless in effect, since the file is removed immediately, but wrong in principle, because the guard wrote to a directory it then told the user it would not touch, and it raced with anything else reading that directory. The probe now runs only after the path rules have failed to produce a refusal: for a directory refused by name the extra reason would not change the outcome, and there is no sense writing into somewhere already declined. For an ordinary directory it still runs, which is the case where being told up front is worth a file created and removed. A unit test and an integration test pin the new ordering.

This is the most instructive of the three. It was invisible to every local run, it was found by an assertion written to check something else, and the honest fix was to change the system rather than to relax the test.

10.9 Defects found by the work itself

Three pre-existing defects surfaced while implementing the above and are worth recording, since each is a plausible bug in comparable tools:

1. `-include-hidden` un-skipped Windows SYSTEM-attributed files along with hidden ones, because the two attributes were tested as one mask.
2. The directory walk aborted an entire scan on any failure below the root.
3. Permission denial below the root was reported as the undifferentiated word “unreadable”, discarding the error kind.

A fourth defect was introduced and caught by the new tests rather than by review: the first integration test for `distribute` destination guarding failed because the sandbox directory lies inside the home directory on Windows, so the pre-existing overlap check rejected the pair before the guard was reached. The test was correct and the fixture was wrong, which is the desirable ordering.

11 Discussion

11.1 Threats to validity

Platform coverage of the probes. Rule tables are validated on every host, but the probe implementations are inherently platform-specific and were executed only on Windows during development. Continuous integration runs the suite on all three platforms; the macOS `st_flags` path and the Linux `mountinfo` live-read path have not been exercised by the author directly. The `mountinfo` *parser* is tested against fixtures on every host, which isolates the untested surface to the file read itself.

Soundness is empirical, not proved. P1 is a claim about a hand-maintained table, not a theorem. New operating system releases introduce directories; the mitigation is the layered composition of name rules, environment variables, runtime attributes and the writability probe, such that a path missing from the table is often still caught by a fact. We make no completeness claim.

Precision is measured against a curated corpus. The 24-directory negative corpus reflects the author’s judgement about what is ordinary. A field study of real invocations would be a stronger instrument and was not conducted.

Time-of-check-to-time-of-use. The classifier judges a directory that could in principle be replaced before the operation proceeds. We consider this acceptable: the guard defends against user error, not against a local adversary who could simply run the tool themselves.

11.2 What the delegation argument does and does not claim

We are not claiming that an agent driving this tool is safe in general. We are claiming something narrower and, we think, checkable: that of the two available interface shapes, one admits a class of attack the other cannot express, and that the stronger one costs little when a plan/execute split already exists.

Three limits are worth stating. The argument concerns the *interface*, not the agent: an agent authorized to apply plans can still apply a reasonable-looking plan for a bad reason, and the defence against that is the review surface and the undo journal rather than the interface shape. The confinement root is resolved once at startup, so a directory replaced beneath a long-running server is a time-of-check hazard of the same kind noted in Section 11. And the server holds plans in memory, so a restart invalidates outstanding handles: correct, but it makes the handle a session capability rather than a durable one, which an orchestrator would need to revisit.

11.3 Generality

The mechanisms are not specific to file organizers. The pure-classifier-plus-facts structure applies to any tool that must judge a target before acting on it; the platform-as-parameter technique applies to any cross-platform path logic and is, we think, underused; the survival taxonomy with one stated fatal exception applies to any bulk operation; and the observation that a move journal cannot express directory deletion applies to any reversible operation that removes containers.

11.4 An orchestrator, and why it is not here

The natural next step, running tidy-up across a fleet from one client, was designed and deliberately not built. A tool that moves files as a privileged user and is reachable over a network is close to remote code execution, and the security model is the architecture rather than a layer over it. Two decisions would be load-bearing: agents should poll for work rather than accept connections, removing the inbound surface entirely; and policy must live on the agent, so that a job says “run profile *photos*” and the agent decides what that means and where it may apply. A request that can name a path turns the whole arrangement into the weaker of the two shapes in Section 9.2, fleet-wide.

11.5 Future work

Four directions follow directly. Effective-UID comparison via `rustix` would complete the ownership signal at the cost of one pure-Rust dependency. A TZif parser would remove the time-zone limitation without a date crate. Per-move instrumentation could be added by way of an observer trait on the executor, which was designed but not implemented, on the grounds that the journal already carries the same information. Finally, the negative corpus should be extended empirically rather than by judgement, which would require telemetry the tool deliberately does not collect — an instructive tension between measurement and the project’s privacy stance.

12 Conclusion

Bulk file reorganization is a category of tool whose core mechanism is indifferent to the value of what it operates on. We have described a guard that closes that gap across Windows, macOS and Linux, built around a pure classifier whose platform is an argument, so that all three rule tables are validated from any host; an empirical writability probe that is correct where mode bits, ACLs and System Integrity Protection disagree; a survival architecture with exactly one stated fatal exception; and an idempotent re-grouping operation whose undo restores directory structure, not merely file locations.

We then took the same question one level further: what changes when the caller is not a person. The answer we arrived at is that an interface in which an agent cannot name a path is strictly stronger than one that validates the paths it names, because the second is only as complete as its author’s list of ways a path can escape, and Section 4 is evidence that such lists are long. A plan/execute split kept for testability turned out to supply exactly the unforgeable handle this needs, which is a pleasant accident worth noticing: the property was available years before the reason to want it.

Three lessons generalize beyond the artifact. The first is that precision is a safety property: a guard that fires on `D:\Downloads` teaches users to bypass it and protects nobody, which is why two plausible rules were deliberately weakened and why a negative corpus is part of the test suite rather than an afterthought. The second is that a journal is a record of a transition, so it cannot answer questions about a state: “is this still undoable” and “what was left alone” both need something the journal structurally does not hold, which is why the review surface is a journal plus a live check plus a log. The third is that separating a pure verdict from an impure policy is what made the work testable at all — and that the same separation forced the discovery, in the Rust standard library, that `std::path` cannot be used to reason about another platform’s paths.

The implementation is 10,265 lines, adds no dependencies, and passes 455 tests on every platform it ships for and on its declared minimum compiler.

Availability

The implementation is open source at <https://github.com/ianktoo/tidy-up>, released under the MIT licence together with a supplementary end-user licence. The modules described here are `src/safety.rs`, `src/outcome.rs`, `src/regroup.rs`, `src/obs.rs`, `src/api.rs`, `src/mcp.rs`, `src/config.rs`, `src/commands/guard.rs` and `src/commands/show.rs`, with tests in the corresponding `*_tests.rs` files and in `tests/`. The work described here was released as versions 0.2.0 and 0.3.0.

Acknowledgements

Portions of the implementation and this manuscript were prepared with the assistance of an AI coding assistant; all design decisions, trade-offs and the final text are the author’s own.

References

- [1] J. B. Dennis and E. C. Van Horn. Programming semantics for multiprogrammed computations. *Communications of the ACM*, 9(3):143–155, 1966.
- [2] M. S. Miller. *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*. PhD thesis, Johns Hopkins University, 2006.
- [3] N. Hardy. The confused deputy (or why capabilities might have been invented). *ACM SIGOPS Operating Systems Review*, 22(4):36–38, 1988.

- [4] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz. Not what you’ve signed up for: compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023.
- [5] Anthropic. *Model Context Protocol specification*. <https://modelcontextprotocol.io>.
- [6] JSON-RPC Working Group. *JSON-RPC 2.0 Specification*, 2010.
- [7] IEEE and The Open Group. *The Open Group Base Specifications Issue 7, 2018 edition (POSIX.1-2017)*. IEEE Std 1003.1-2017. See in particular `access()` and `faccessat()`.
- [8] M. Kerrisk. *The Linux Programming Interface*. No Starch Press, 2010.
- [9] Linux kernel documentation. *proc(5) — process information pseudo-filesystem*. Describes the `/proc/[pid]/mountinfo` format, including octal escaping of mount points and the optional-field separator.
- [10] Linux Foundation. *Filesystem Hierarchy Standard, Version 3.0*, 2015.
- [11] Apple Inc. *System Integrity Protection Guide*. Apple Developer Documentation.
- [12] Apple Inc. *Apple File System Reference*. Apple Developer Documentation. Describes firmlinks and the split System/Data volume layout.
- [13] Microsoft Corporation. *Naming Files, Paths, and Namespaces*. Windows Application Development Documentation. Covers reserved device names, trailing dot and space handling, and the `\\?\` prefix.
- [14] Microsoft Corporation. *File Attribute Constants*. Windows Application Development Documentation.
- [15] Microsoft Corporation. *Known Folders and the `KNOWNFOLDERID` constants*. Windows Application Development Documentation.
- [16] GNU Project. *GNU Coreutils Manual*. See `rm` and the `-preserve-root` option.
- [17] H. Hinnant. *chrono-Compatible Low-Level Date Algorithms*. Describes the `civil_from_days` algorithm used for date conversion.
- [18] The Rust Project Developers. *The Rust Standard Library Documentation*. See `std::path`, `std::io::ErrorKind`, and the platform `MetadataExt` extension traits.
- [19] I. Too. *tidy-up: organize messy folders by file type, isolate duplicates, and undo it all*. <https://github.com/ianktoo/tidy-up>.